RBAC Matrix

Ref: 3.1
Product: iConnect Plus
Organisation: Cache Digitech
Classification: Internal — Confidential
Version: 2.0
Date: 2026-11-03
Deployment model: On-premises (Cache Digitech premises) · VPN for remote access
Audience: Security assessors, tenant admins, module leads, auditors

---

1. Purpose

Describe the **role-based access control (RBAC)** model for iConnect Plus: permission naming, system roles, module roles, and a representative matrix. The authoritative permission list is the **aggregated catalog** in code and database.

---

2. RBAC model

Concept	Storage / SSOT
Permission	Code: `apps/api/src/modules/*/permissions.py` → DB: `sec_permission`
Role	`sec_role` per tenant + system role packs
Assignment	User ↔ role ↔ tenant
Enforcement	API: `require_permission(code)`; UI: route guards vs `modules.ts`
Cache	Redis-backed resolution in `RBACService`

Permission code format: `{module}.{resource}:{action}`
Examples: `foundation.user:read`, `hr.employee:update`, `asset.component:create`.

---

3. System roles (platform-wide)

Role code	Name	Scope	Typical permissions
`SUPER_ADMIN`	Super Admin	Cross-tenant platform ops	Full foundation + tenant administration
`TENANT_ADMIN`	Tenant Admin	Single tenant	User/role/setting management within tenant

Defined in `modules/foundation/permissions.py` (`SYSTEM_ROLE_PACKS`).

---

4. Module workspace roles (pattern)

Many modules define **workspace role packs** (e.g. HR_EMPLOYEE, HR_MANAGER, HR_ADMIN) mapping to subsets of module permissions. Pattern:

Layer	Example (HR)	Capability
Employee self	`HR_EMPLOYEE`	Own data, limited reads
Line manager	`HR_MANAGER`	Team approvals, broader read
Module admin	`HR_ADMIN`	Full module CRUD within tenant

Each module maintains its own pack in `{module}/permissions.py`.

5. Representative permission matrix — Foundation

Permission code	Super Admin	Tenant Admin	Standard user
`foundation.tenant:read`	Yes	Yes (own)	No
`foundation.tenant:create`	Yes	No	No
`foundation.user:read`	Yes	Yes	No
`foundation.user:create`	Yes	Yes	No
`foundation.role:read`	Yes	Yes	No
`foundation.role:update`	Yes	Yes	No
`foundation.workflow:approve`	Yes	Yes	As assigned
`foundation.audit:read`	Yes	Yes	No
`foundation.setting:update`	Yes	Yes	No

Standard users receive **module-specific** roles only — not tenant administration.

6. Representative permission matrix — HR (illustrative)

Permission area	Employee	Manager	HR Admin
Own profile / ESS	Yes	Yes	Yes
Team attendance view	No	Yes	Yes
Employee master CRUD	No	Limited	Yes
Payroll run	No	No	Yes (if licensed)
`hr.superadmin:manage`	No	No	No (platform only)

Exact codes: see `apps/api/src/modules/hr/permissions.py`.

7. Segregation of duties (SoD) — template

Conflict	Role A	Role B	Mitigation
Create vendor + approve payment	AP clerk	Finance approver	Separate permissions
HR hire + payroll approve	HR exec	Payroll admin	Workflow steps
Security admin + audit only	Tenant admin	Auditor read-only	**Template / Pending sign-off**

Module-specific SoD rules documented in FRDs and ADRs.

8. Tenant isolation

Rule	Enforcement
Users belong to tenant(s)	Auth context
Roles scoped per tenant	DB constraints
Queries filter `tenant_id`	Repository layer (CI reviewed)

9. CI / governance gates

Gate	Description
Catalog sync	Migration syncs permissions to `sec_permission`
Route ⊆ catalog	Architecture test
Web ⊆ catalog	`modules.ts` alignment

10. Operational procedures

Procedure	Frequency	Evidence
Permission catalog review on release	Each release	PR + migration
Role assignment review	Quarterly	§3.3
Privileged role inventory	Quarterly	**Template / Pending sign-off**

11. Full catalog export

Export all permission codes:

```
# From repo root — run on internal dev machine
python -c "from modules.platform.permission_catalog import iter_all_permissions; ..."
```

Or enable `ENABLE_API_DOCS` on staging and inspect seeded data via admin UI — **Template / Pending sign-off**.

12. Related documents

Ref	Document
3.2	Authentication and Session Design
3.3	User Access Review Records
1.6	API Specification

Approval: Template / Pending sign-off